AgroStock

Entrega 2
Ingeniería de Software

Equipo

Juan Felipe Cardona
Juan Camilo González
Daniel Villa

2026-10-12

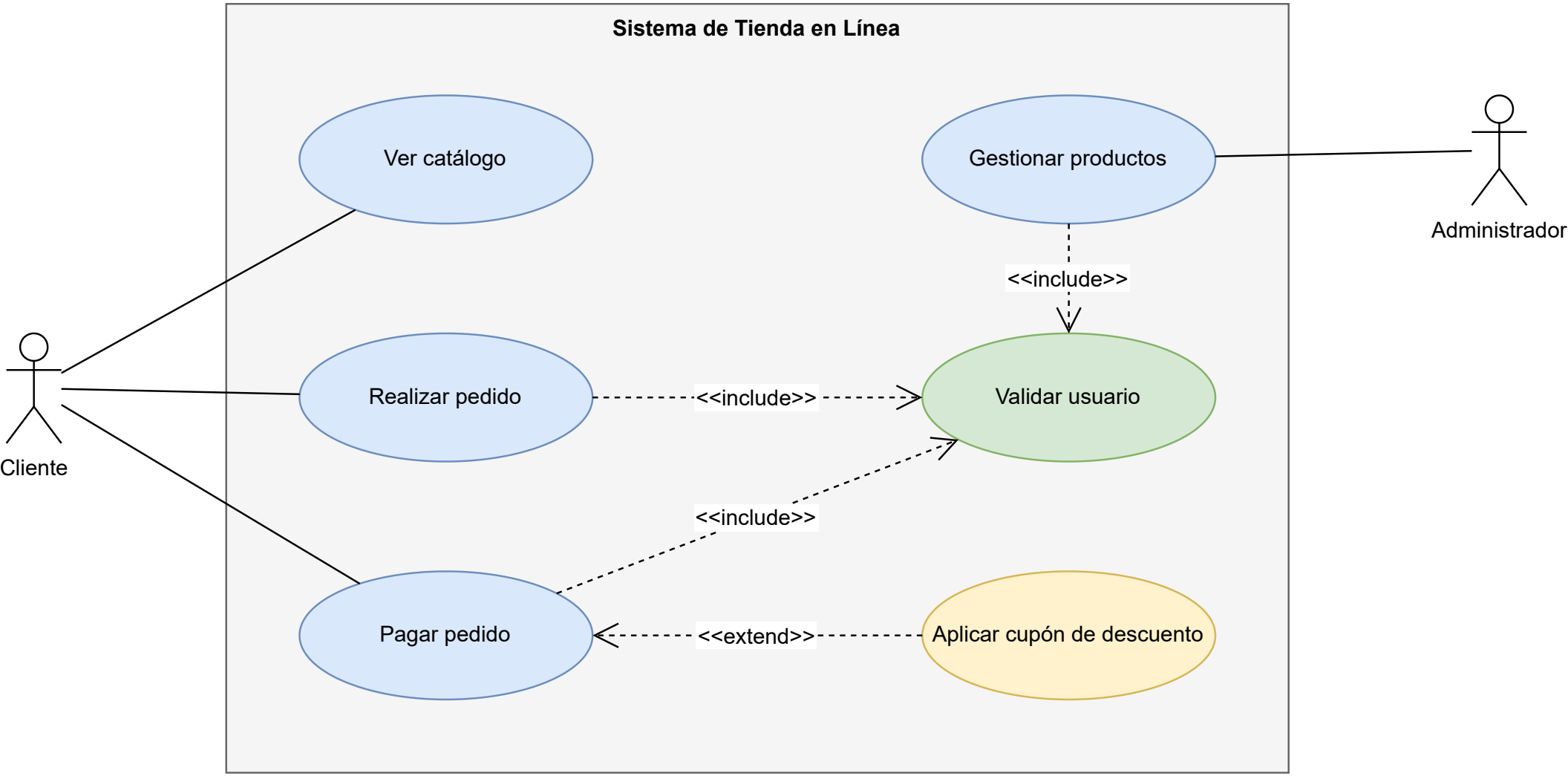
Repositorio: Camilogm1/Agrostock, rama Daniel

Índice

1. Modelos UML y C4
2. Decisiones de arquitectura (ADR)
3. Matriz de decisión del stack
4. Plan de pruebas
5. Prototipo funcionando
6. Registro de deuda técnica
7. Matriz de riesgos actualizada
8. Tabla de trazabilidad
9. Nota de uso de IA
10. Documento consolidado y fuentes

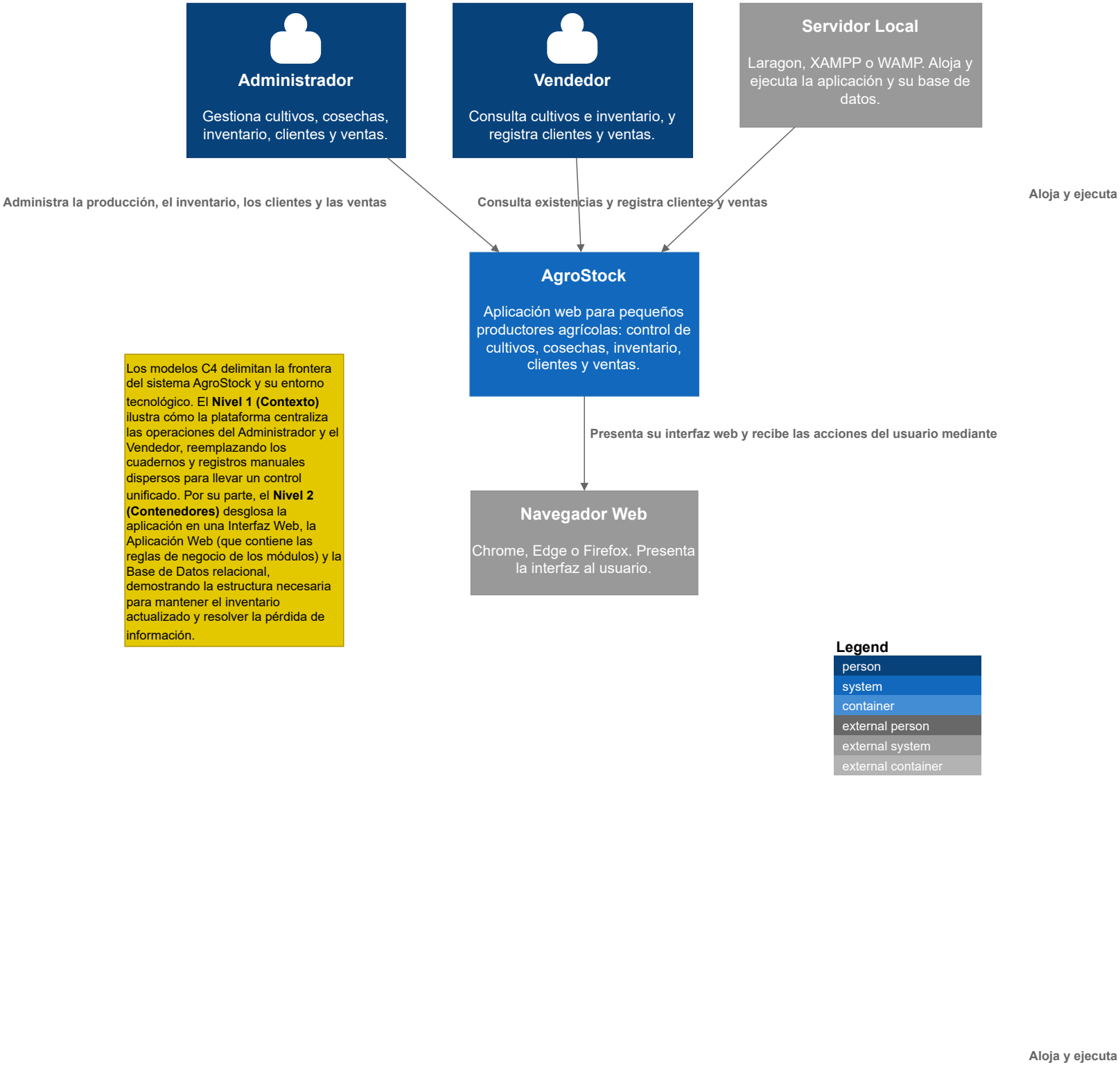
1. Modelos UML y C4

El taller solicita seis modelos: caso de uso, clases, secuencia, componentes y despliegue, C4 nivel 1 y C4 nivel 2. Se encontraron cinco diagramas renderizados, listados abajo. Aunque los SVG incluyen una pestaña llamada 'Diagrama de clases', su XML no contiene elementos de clase ni relaciones; falta la fuente visual de ese modelo para completar este requisito.



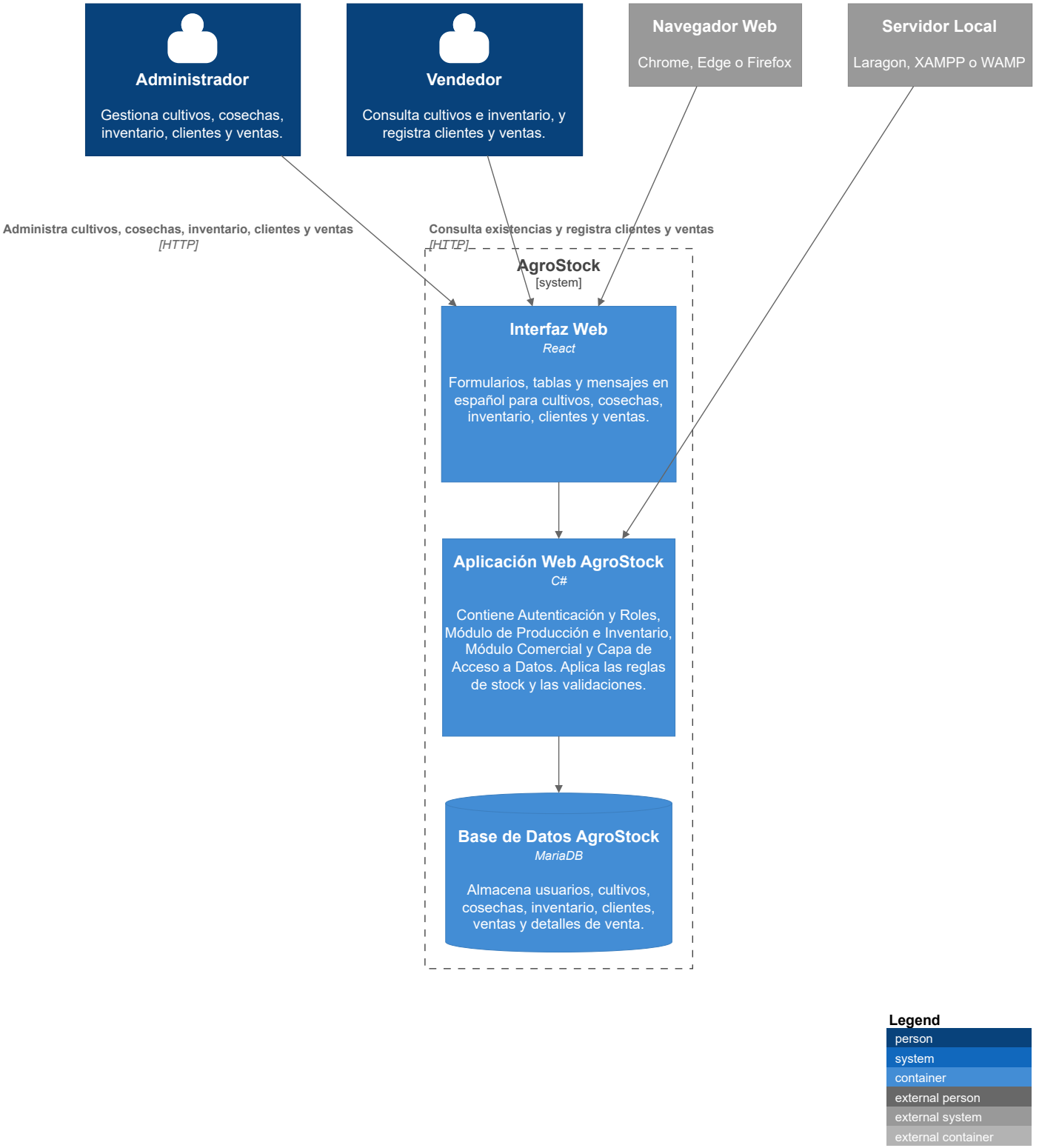
Este diagrama representa las interacciones del Administrador y el Vendedor con el Módulo Comercial. Se estructuró utilizando relaciones **<<include>>** desde 'Registrar Venta' hacia 'Seleccionar Cliente y Producto', 'Validar Stock Disponible' y 'Descontar Inventario', ya que estas acciones son pasos obligatorios para completar una venta correctamente y evitar vender productos sin existencias, resolviendo así una de las problemáticas principales del negocio. Además, se emplearon relaciones **<<extend>>** para 'Registrar Cliente (cliente nuevo)' y 'Buscar Venta', dado que son flujos alternativos u opcionales que extienden el comportamiento base solo cuando es necesario.

Modelo 5 – C4 Nivel 1 (Contexto) – AgroStock

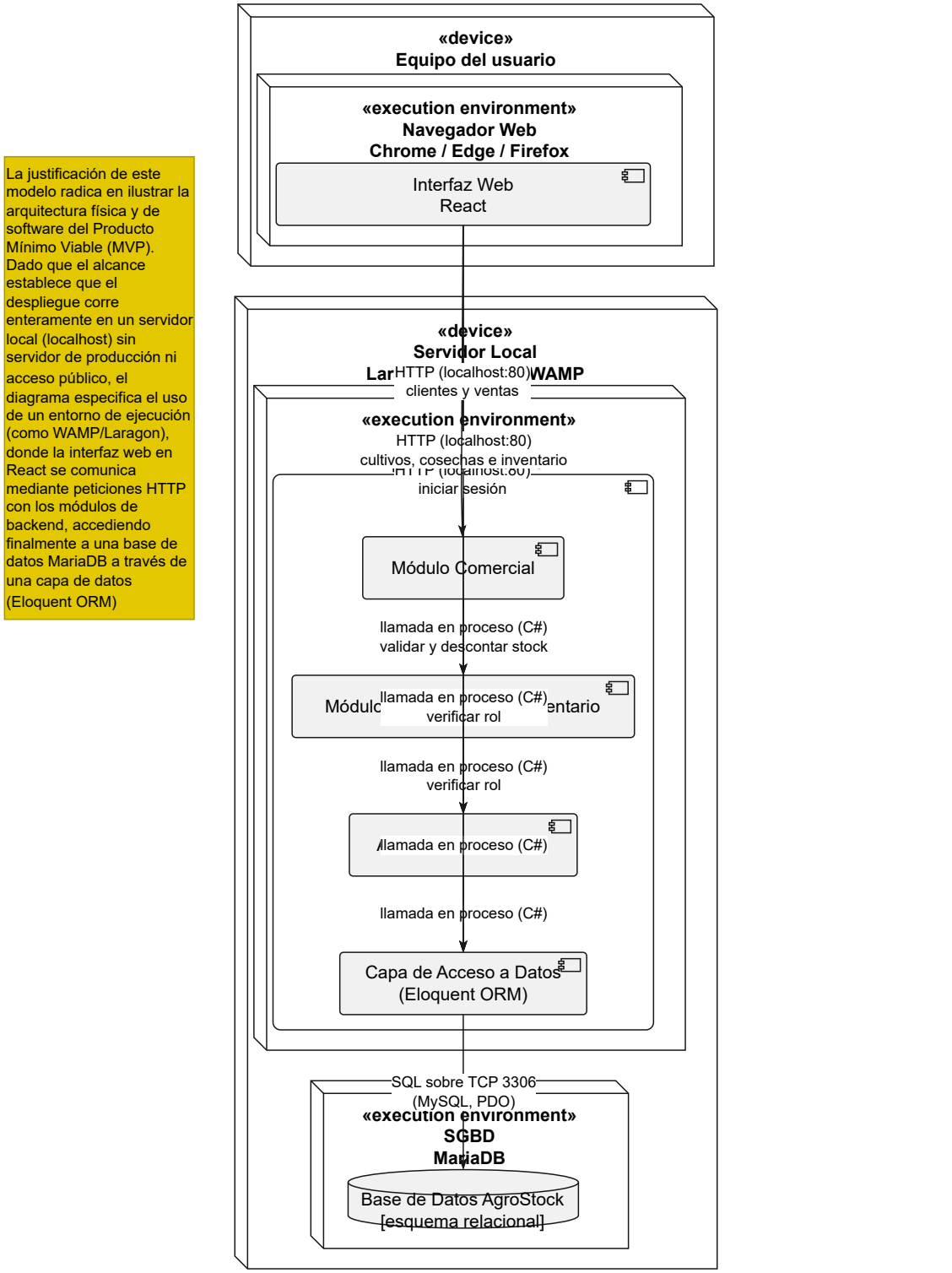


Los modelos C4 delimitan la frontera del sistema AgroStock y su entorno tecnológico. El **Nivel 1 (Contexto)** ilustra cómo la plataforma centraliza las operaciones del Administrador y el Vendedor, reemplazando los cuadernos y registros manuales dispersos para llevar un control unificado. Por su parte, el **Nivel 2 (Contenedores)** desglosa la aplicación en una Interfaz Web, la Aplicación Web (que contiene las reglas de negocio de los módulos) y la Base de Datos relacional, demostrando la estructura necesaria para mantener el inventario actualizado y resolver la pérdida de información.

Modelo 6 – C4 Nivel 2 (Contenedores) – AgroStock



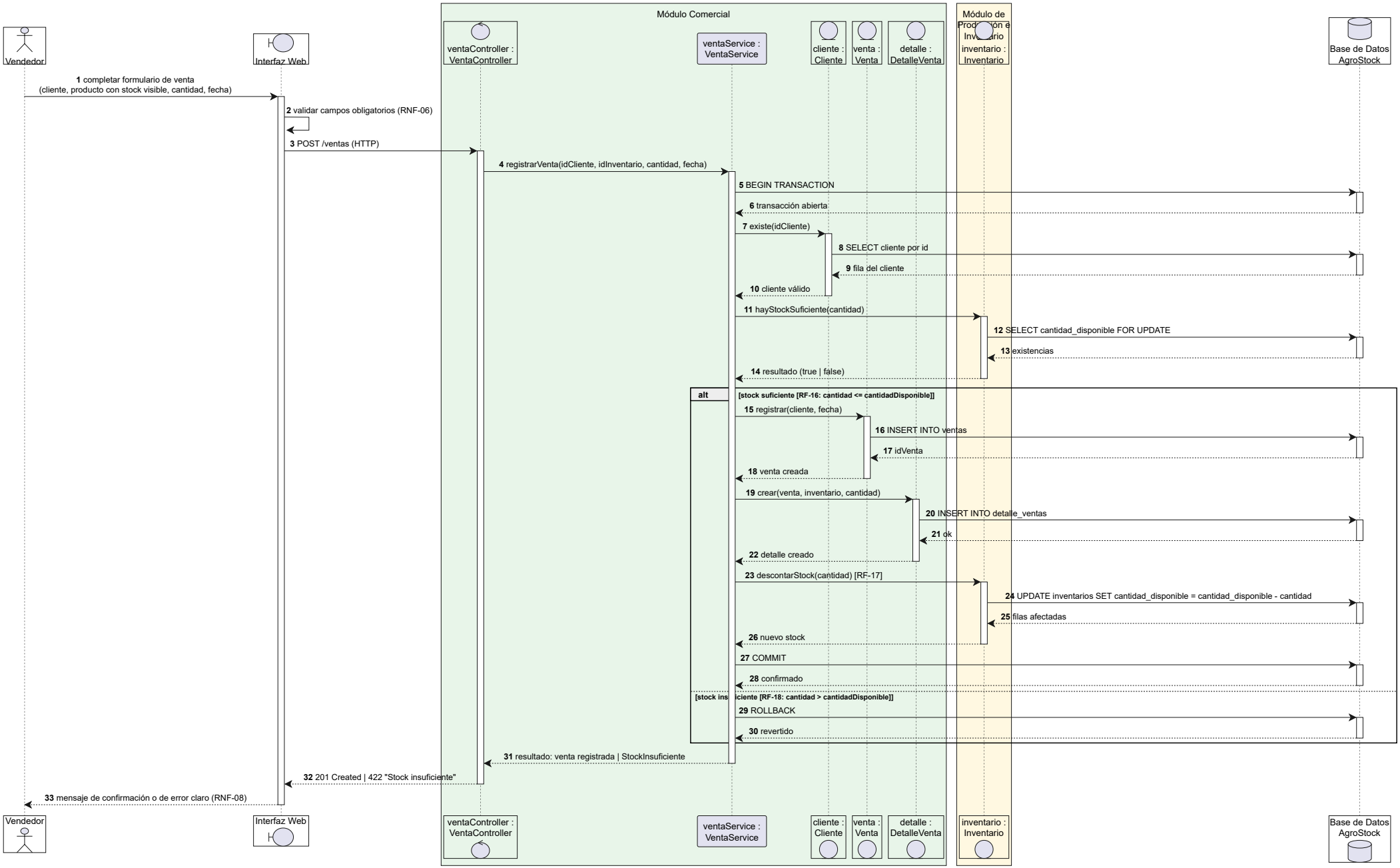
Modelo 4 – Diagrama de componentes y despliegue – AgroStock



La justificación de este modelo radica en ilustrar la arquitectura física y de software del Producto Mínimo Viable (MVP). Dado que el alcance establece que el despliegue corre enteramente en un servidor local (localhost) sin servidor de producción ni acceso público, el diagrama especifica el uso de un entorno de ejecución (como WAMP/Laragon), donde la interfaz web en React se comunica mediante peticiones HTTP con los módulos de backend, accediendo finalmente a una base de datos MariaDB a través de una capa de datos (Eloquent ORM)

Todo el despliegue corre en localhost: sin servidor de producción ni acceso público (Entrega 1, sección 2.4).

Modelo 3 – Secuencia – Registrar venta con validación de stock y descuento de inventario



Este diagrama detalla el flujo cronológico y la lógica transaccional requerida para registrar una venta. Justifica cómo el sistema aborda la problemática de los errores de inventario mediante un bloque transaccional (Transaction) en la base de datos: primero valida que exista suficiente stock mediante un `SELECT`, luego inserta los datos de la venta y el detalle, y finalmente descuenta la cantidad del inventario con un `UPDATE`. Esto garantiza la consistencia e integridad de los datos si ocurre un error a mitad del proceso

2. Decisiones de arquitectura (ADR)

• Estado: Aceptada para el prototipo actual

• Fecha: 2026-10-12

Contexto

El repositorio contiene una API para el taller de CI, no la aplicación completa descrita por los modelos. La solución apunta a .NET 8, usa ASP.NET Core, EF Core InMemory y xUnit. El README indica que esta versión es autocontenida y que no instala un motor de base de datos. Los diagramas y el script SQL, en cambio, describen React, Laravel/Eloquent y MariaDB.

Decisión

Se conserva el stack que ya compila en CI para ampliar el prototipo evaluable: ASP.NET Core sobre .NET 8, EF Core InMemory y xUnit. Esta decisión limita el alcance al prototipo del repositorio y no declara que sea el stack de producción de AgroStock.

Alternativas descartadas

• Migrar ahora a React, Laravel/Eloquent y MariaDB: coincide con los diagramas, pero reemplaza la implementación existente y requiere configurar otro runtime y motor de datos para desarrollar y probar.

• Incorporar MariaDB a la API actual: permite persistencia real, pero no está configurado en el proyecto ni en el pipeline actual.

Consecuencias

• Positivas: se reutiliza el código y el pipeline existente; las pruebas pueden ejecutarse sin un servicio externo.

• Negativas: los datos no persisten al reiniciar la API; EF Core InMemory no valida las restricciones, triggers ni transacciones del script MariaDB; la arquitectura del prototipo difiere de los diagramas.

• Pendiente: tratar la diferencia entre el prototipo y los modelos como una brecha explícita antes de afirmar que el prototipo implementa la arquitectura completa.

Fuentes

• README

• Proyecto API

• Registro de servicios

• Pipeline CI

• Estado: Aceptada para el prototipo actual

• Fecha: 2026-10-12

Contexto

La API expone `GET /api/cultivos` y `POST /api/cultivos`. El controlador delega en `ICultivoService`; `CultivoService` valida los datos y usa `AppDbContext` para persistirlos. No hay otras capas de acceso a datos en el prototipo.

Decisión

Mantener los endpoints en `CultivosController` y las reglas del caso de uso en `CultivoService`, detrás de `ICultivoService`. El contexto de EF Core se inyecta al servicio.

Alternativas descartadas

• Poner validaciones y persistencia directamente en el controlador: reduce archivos al inicio, pero acopla HTTP, reglas de negocio y acceso a datos, y dificulta probar el caso de uso sin pasar por la API.

• Añadir un repositorio genérico entre el servicio y EF Core: agrega una abstracción que no se reutiliza ni aporta comportamiento adicional en este prototipo pequeño.

Consecuencias

• Positivas: las reglas del caso de uso se prueban con xUnit sin iniciar un servidor HTTP; el controlador queda como adaptador HTTP.

• Negativas: el servicio depende directamente de EF Core y el caso de uso está acoplado al modelo de persistencia; al agregar módulos se deberá revisar si conviene separar dominios o transacciones.

Fuentes

• Controlador

• Interfaz del servicio

• Servicio

• Pruebas del servicio

• Estado: Aceptada para el prototipo actual

• Fecha: 2026-10-12

Contexto

El registro de un cultivo en el prototipo crea también una fila de inventario con cantidad disponible igual a cero. La prueba de aceptación de `RegistrarAsync` verifica ese comportamiento. El script `Base de datosV2.txt` describe otro flujo: el inventario se crea o incrementa al registrar una cosecha.

Decisión

Para el prototipo actual, crear una fila de inventario en cero inmediatamente después de guardar el cultivo. Esta regla es parte del caso de uso simplificado y no sustituye el flujo de cosechas definido en el script SQL.

Alternativas descartadas

• Crear el inventario únicamente al registrar la primera cosecha, como en el trigger de `Base de datosV2.txt`: coincide con el flujo del modelo de datos completo, pero requeriría implementar cosechas en esta API.

• No crear inventario al registrar el cultivo y esperar a una operación explícita de inventario: agrega un paso manual y no satisface la prueba existente del prototipo.

Consecuencias

• Positivas: el cultivo recién creado siempre tiene inventario consultable; el comportamiento está cubierto por una prueba.

• Negativas: el registro y la creación del inventario son dos llamadas a `SaveChangesAsync`; con una base relacional, un fallo entre ambas puede dejar datos parciales si no se usa una transacción.

• Negativas: difiere del trigger de MariaDB, que inicializa el inventario en la primera cosecha. La integración de ambos comportamientos requiere una decisión antes de conectar una base persistente.

Fuentes

• Servicio de cultivos

• Pruebas de cultivos

• Script de base de datos v2

3. Matriz de decisión del stack

Criterios y escala

Cada opción recibe una calificación de 1 a 5; 5 es el mejor ajuste. El puntaje total es la suma ponderada. El conocimiento previo pesa doble, como pide la rúbrica.

Criterio	Peso	Interpretación
Lo que el equipo ya sabe	2	Evidencia de uso en este repositorio; no sustituye una encuesta a los integrantes.
Ajuste a los modelos	1	Correspondencia con la arquitectura representada en los diagramas.
Continuidad con el repositorio	1	Reutilización del código y estructura existentes.
Pruebas y CI	1	Facilidad para probar y ejecutar en el pipeline actual.
Sencillez de ejecución local	1	Menos servicios y configuración necesarios para el prototipo.

Las calificaciones de conocimiento se basan solo en evidencia comprobable: el backend .NET ya tiene código y pruebas; no hay frontend ni código PHP o Node en el repo. Por eso, un 1 significa "sin evidencia en este repositorio", no que el equipo desconozca la tecnología. El equipo debe confirmar estas puntuaciones antes de presentar la matriz como encuesta de experiencia.

Frontend

Opción	Conocimiento (x2)	Modelos	Continuidad	CI/pruebas	Ejecución local	Total / 30
React	1 (2)	5	4	4	3	**18**
Vue	1 (2)	2	4	4	4	16
Blazor	1 (2)	1	5	4	5	17

Recomendación: React para una futura interfaz, por ser el frontend que aparece en los modelos. No está implementado en el prototipo actual; Swagger UI es la interfaz disponible para explorar la API.

Por qué se descarta la segunda opción, Blazor: se aparta del frontend React representado en los modelos y añadiría una aplicación web que hoy no existe ni hace falta para demostrar los endpoints de la API.

Backend

Opción	Conocimiento (x2)	Modelos	Continuidad	CI/pruebas	Ejecución local	Total / 30
ASP.NET Core .NET 8 + EF Core InMemory	5 (10)	2	5	5	5	**27**
Laravel + Eloquent + MariaDB	1 (2)	5	1	3	2	13
Node.js + Express	1 (2)	2	1	4	3	12

Decisión para el prototipo: conservar ASP.NET Core .NET 8, EF Core InMemory y xUnit. Es el stack presente, compila en CI y el usuario confirmó que se debe seguir usando. Esta decisión no afirma que el prototipo ya corresponda a la arquitectura Laravel/MariaDB de los modelos.

Por qué se descarta la segunda opción, Laravel: exigiría reemplazar el backend y el pipeline .NET que ya pasan pruebas; además requiere configurar PHP y MariaDB, servicios que no están en el repo ni son necesarios para la demo autocontenida actual.

Alcance y validación pendiente

La recomendación React es una evaluación para una interfaz futura, no una dependencia agregada. Los puntajes de familiaridad de todo el equipo deben confirmarse con sus integrantes; el repositorio por sí solo solo demuestra que existe código y experiencia de trabajo compartido en .NET.

Fuentes: README, API, pruebas, CI.

4. Plan de pruebas

Pirámide de pruebas

Nivel	Qué se verifica	Estado en este repositorio
Unitarias	Validaciones y reglas de `CultivoService` aisladas con EF Core InMemory.	7 casos automatizados.
Integración	Rutas HTTP, códigos de respuesta, serialización, inyección y persistencia a través de la API.	No hay pruebas automatizadas.
Sistema / extremo a extremo	Recorrido de usuario por una interfaz web y persistencia en una base real.	No hay frontend ni pruebas automatizadas de este nivel.

La CI de GitHub Actions restaura, compila y ejecuta las pruebas unitarias. El uso de EF Core InMemory no sustituye una prueba de integración con MariaDB ni valida sus triggers.

Pruebas implementadas

Las siete ejecuciones corresponden a cinco métodos de prueba: una prueba de registro, tres casos de una teoría de validación, una prueba de listado y dos pruebas de consulta individual.

Prueba	Nivel	Resultado esperado
`RegistrarAsync_ConDatosValidos_CreaCultivoConInventarioInicialEnCero`	Unitaria	Asigna ID, conserva nombre y crea inventario inicial en cero.
`RegistrarAsync_ConCampoObligatorioVacio_NoRegistraDatos` con nombre vacío	Unitaria	Lanza `ArgumentException`; no crea cultivo ni inventario.
`RegistrarAsync_ConCampoObligatorioVacio_NoRegistraDatos` con tipo en blanco	Unitaria	Lanza `ArgumentException`; no crea cultivo ni inventario.
`RegistrarAsync_ConCampoObligatorioVacio_NoRegistraDatos` con lote vacío	Unitaria	Lanza `ArgumentException`; no crea cultivo ni inventario.
`ListarAsync_DespuesDeRegistrarCultivos_DevuelveTodosLosRegistrados`	Unitaria	Devuelve los cultivos registrados.
`ObtenerPorIdAsync_ConCultivoRegistrado_DevuelveElCultivo`	Unitaria	Devuelve el cultivo solicitado por identificador.
`ObtenerPorIdAsync_SinCoincidencia_DevuelveNull`	Unitaria	Devuelve `null` cuando no existe el identificador.

Fuente: pruebas xUnit.

Trazabilidad a criterios de aceptación

No se encontró el backlog formal en el repositorio. Los IDs siguientes son referencias provisionales reconstruidas de los comentarios y pruebas existentes; deben cotejarse con las historias oficiales antes de entregar.

Referencia provisional	Dado	Cuando	Entonces	Prueba
RF-01 / AC-01	Un administrador dispone de nombre, tipo, lote y fecha válidos.	Registra un cultivo.	El cultivo se crea y se devuelve con identificador.	`RegistrarAsync_ConDatosValidos_CreaCultivoConInventarioInicialEnCero`

Referencia provisional	Dado	Cuando	Entonces	Prueba
RNF-06 / AC-02	El nombre, tipo o lote está vacío o en blanco.	Se intenta registrar el cultivo.	La operación falla con `ArgumentException` y no guarda cultivo ni inventario.	`RegistrarAsync_ConCampoObligatorioVacio_NoRegistraDatos` (3 datos)
RF-07 / AC-03	Se registra un cultivo válido.	El servicio termina el registro.	Existe inventario asociado con cantidad disponible igual a cero.	`RegistrarAsync_ConDatosValidos_CreaCultivoConInventarioInicialEnCero`
RF-02 / AC-04	Hay cultivos registrados.	Se solicita el listado.	La respuesta contiene todos los cultivos guardados.	`ListarAsync_DespuesDeRegistrarCultivos_DevuelveTodosLosRegistrados`
RF-02 / AC-05	Existe o no un cultivo con el identificador consultado.	Se solicita `GET /api/cultivos/{id}`.	El servicio devuelve el cultivo o `null`; el controlador responde `200` o `404`, respectivamente.	`ObtenerPorIdAsync_ConCultivoRegistrado_DevuelveElCultivo`; `ObtenerPorIdAsync_SinCoincidencia_DevuelveNull`; códigos HTTP solo verificados manualmente

Verificación manual disponible

Swagger permite probar `POST /api/cultivos` y `GET /api/cultivos` al ejecutar la API localmente. Este recorrido es una comprobación manual, no se cuenta como prueba automatizada ni reemplaza pruebas de integración. Véanse las instrucciones del README.

Nota de alcance: la prueba RF-07 refleja el comportamiento simplificado del prototipo. El trigger en `Base de datosV2.txt` crea el inventario al registrar la primera cosecha; esa diferencia debe resolverse al integrar ambos modelos.

5. Prototipo funcionando

Versión mínima y autocontenida para el taller evaluable de integración continua.

No es el proyecto completo de AgroStock (ese tiene autenticación, MariaDB, y los

módulos de Cosechas/Inventario/Clientes/Ventas). Aquí solo vive la historia RF-01,

lo mínimo necesario para cumplir los 5 entregables del taller.

Stack

• C# / ASP.NET Core Web API (.NET 8)

• Entity Framework Core (InMemory — no requiere instalar ninguna base de datos)

• xUnit

Cómo levantar el proyecto

```bash

cd src/AgroStockTaller.Api

dotnet restore

dotnet run

```

Abre `http://localhost:5000/swagger` (o el puerto que indique la consola) para probar

`POST /api/cultivos`, `GET /api/cultivos` y `GET /api/cultivos/{id}` manualmente.

Nota importante para Windows: si el repositorio está dentro de OneDrive, el ensamblado de pruebas puede quedar bloqueado por la protección de "Control de aplicaciones". Para evitarlo, clona o copia el proyecto en una carpeta normal como `C:\Users\<tu_usuario>\source\repos\Agrostock`.

Cómo correr las pruebas

```bash

cd tests/AgroStockTaller.Tests

dotnet restore

dotnet test

```

Debe mostrar 7 casos aprobados y ninguno fallido.

Estado del pipeline

!CI

Documentación de entrega

• ADR

• Matriz de decisión del stack

• Plan de pruebas

• Registro de deuda técnica

• Matriz de riesgos

• Trazabilidad

• Nota de uso de IA

• Modelos editables

• Documento consolidado en PDF

Al contrastar los modelos solicitados con las fuentes recibidas, no se encontró

un diagrama de clases renderizado: la pestaña con ese nombre en los SVG no

contiene clases ni relaciones. Está registrado como pendiente en la trazabilidad.

Para regenerar el PDF en Windows, instala las dependencias con

`py -m pip install -r docs/requirements-pdf.txt` y ejecuta

`py scripts/build_delivery_pdf.py`. Se requiere Microsoft Edge o Google Chrome.

Integrantes

• Juan Felipe Cardona

• Juan Camilo González

• Daniel Villa

6. Registro de deuda técnica

Estimaciones preliminares en días-persona para una persona familiarizada con el repo; deben revisarse con el equipo. Los cuadrantes combinan deliberada/inadvertida y prudente/imprudente.

ID	Deuda y evidencia	Cuadrante	Costo estimado	Plan de pago
DT-01	El prototipo usa EF Core InMemory: los datos se pierden al reiniciar y no se ejercitan las restricciones ni triggers de MariaDB.	Deliberada / prudente: simplifica la demo autocontenida, pero limita su uso como aplicación.	2-3 días	Cuando se autorice persistencia real, agregar un proveedor relacional, configuración local y pruebas de integración; conservar InMemory solo para pruebas unitarias si sigue aportando valor.
DT-02	La implementación es .NET y la API solo cubre cultivos; los modelos describen React, Laravel/Eloquent, MariaDB y otros módulos.	Deliberada / imprudente para una entrega que pretenda representar el producto completo: la diferencia ya es conocida y queda visible en ADR-0001.	3-5 días	Confirmar alcance con el equipo; si el stack del repo es el definitivo, actualizar los modelos en una revisión posterior; si los modelos mandan, planificar migración por módulos y no declarar equivalencia antes de implementarla.
DT-03	`RegistrarAsync` guarda cultivo e inventario en llamadas separadas a `SaveChangesAsync`; una falla en la segunda puede dejar estado parcial con un proveedor relacional.	Inadvertida / imprudente: no hay transacción en la operación.	0.5-1 día	Al añadir persistencia relacional, envolver ambas escrituras en una transacción y probar rollback con una base relacional. EF Core InMemory no permite validar esta propiedad.

La clasificación es una lectura del estado actual, no una decisión histórica del equipo. Fuentes: ADR-0001, ADR-0003, servicio, proyecto API y script MariaDB.

7. Matriz de riesgos actualizada

Fecha de revisión: 2026-10-12. No se encontró en el repositorio la matriz original de la semana 8. Esta revisión registra evidencia disponible ahora; no permite reconstruir ni atribuir estados previos a riesgos que no estén documentados.

Escala: probabilidad e impacto de 1 (bajo) a 5 (alto); exposición = probabilidad × impacto.

ID	Riesgo / evidencia	Prob.	Impacto	Exposición	Estado	Respuesta
R-01	La arquitectura del prototipo .NET difiere de los modelos React/Laravel/MariaDB.	5	5	25	Materializado	Mantener explícito que .NET es el stack del prototipo; acordar si los diagramas representan el producto futuro antes de declarar cumplimiento integral.
R-02	EF Core InMemory pierde los datos al reiniciar y no reproduce el comportamiento de MariaDB.	5	4	20	Nuevo riesgo técnico	No usar la demo como persistencia real; añadir proveedor relacional y pruebas de integración si el alcance lo requiere.
R-03	El caso de uso de registro puede guardar cultivo sin inventario si falla la segunda escritura con una base relacional.	3	5	15	Nuevo riesgo técnico	Hacer ambas escrituras atómicas y probar rollback con proveedor relacional.
R-04	El prototipo solo expone registro y listado de cultivos; no demuestra cosechas, ventas, clientes ni autenticación de los modelos.	5	5	25	Brecha de alcance vigente	Priorizar historias del backlog y ampliar por módulo; no presentar la API actual como producto completo.
R-05	Sin pruebas de integración, los contratos HTTP y la configuración real de almacenamiento pueden romperse sin detección automática.	4	4	16	Nuevo riesgo técnico	Incorporar pruebas de API con servidor de prueba y almacenamiento adecuado cuando se definan los contratos.
R-06	Que un cambio se integre sin compilar o ejecutar pruebas.	2	4	8	Cerrado en el repo; Actions remoto por verificar	Existe workflow para push y pull request que restaura, compila y ejecuta pruebas. La ejecución local fue exitosa; verificar que el último run remoto esté verde antes de entregar.

ID	Riesgo / evidencia	Prob.	Impacto	Exposición	Estado	Respuesta
R-07	La pestaña "Diagrama de clases" de los SVG recibidos contiene una anotación, pero no clases ni relaciones renderizables.	5	4	20	Brecha materializada en los artefactos disponibles	Solicitar al equipo el archivo fuente del diagrama de clases e incorporarlo al PDF y a la trazabilidad.

R-06 se considera cerrado respecto a la ausencia de automatización porque el workflow está versionado en CI. No afirmo que otros riesgos históricos de semana 8 estén cerrados: falta su matriz fuente y no se consultó el resultado remoto de GitHub Actions.

8. Tabla de trazabilidad

Los IDs HU/AC de esta tabla son provisionales: el backlog oficial no está versionado en el repositorio. Los cinco SVG de docs/modelos contienen pestañas draw.io con los nombres de los seis modelos solicitados, pero la pestaña "Diagrama de clases" solo tiene una anotación, sin elementos de clase ni relaciones. Hace falta esa fuente visual. Antes de entregar, el equipo debe cotejar estos IDs y nombres con su backlog original.

Historia / requisito	Modelo donde aparece	Prueba que lo verifica	Módulo o archivo de código	Estado
RF-01 / AC-01: registrar un cultivo con datos válidos	Diagrama de clases requerido, pero no visible en las fuentes recibidas	`RegistrarAsync_ConDatosValidos_CreaCultivoConInventarioInicialEnCero`	`CultivosController`, `CultivoService`, `Cultivo`, `CultivoRequest`	Implementado en el prototipo; falta el modelo visual para completar la trazabilidad.
RNF-06 / AC-02: rechazar nombre, tipo o lote vacío	El modelo de clases no está disponible visualmente; las fuentes incluyen solo una anotación en su pestaña	`RegistrarAsync_ConCampoObligatorioVacio_NoRegistraDatos` (3 casos)	`CultivoService.RegistrarAsync`	Implementado y probado a nivel de servicio; relación con modelo pendiente.
RF-02 / AC-04: listar cultivos registrados	El modelo de clases no está disponible visualmente; el caso de uso disponible no representa explícitamente esta consulta	`ListarAsync_DespuesDeRegistrarCultivos_DevuelveTodosLosRegistrados`	`GET /api/cultivos`, `CultivoService.ListarAsync`	Implementado y probado a nivel de servicio; relación con modelo pendiente.
RF-02 / AC-05: consultar un cultivo por identificador	El modelo de clases no está disponible visualmente; no aparece como flujo explícito en el caso de uso	`ObtenerPorIdAsync_ConCultivoRegistrado_DevuelveElCultivo`; `ObtenerPorIdAsync_SinCoincidencia_DevuelveNull`	`GET /api/cultivos/{id}`, `CultivoService.ObtenerPorIdAsync`	Implementado; respuesta HTTP comprobable manualmente, sin prueba automatizada del controlador.
RF-07 / AC-03: iniciar el inventario de un cultivo en cero	El modelo de clases no está disponible visualmente; la secuencia recibida trata ventas	`RegistrarAsync_ConDatosValidos_CreaCultivoConInventarioInicialEnCero`	`CultivoService.RegistrarAsync`, `Inventario`, `AppDbContext`	Implementado en el prototipo; difiere del trigger del SQL v2, que crea inventario al registrar cosecha.
RF-04: impedir eliminar cultivos con cosechas	El modelo de clases requerido no está disponible visualmente; script SQL v2 define FK y trigger	No existe	No hay entidad Cosecha ni endpoint de eliminación	Diseñado en SQL; no implementado ni probado en la API.
RF-16/RF-17: verificar stock y descontar inventario en una venta	Diagrama de caso de uso y secuencia; trigger en SQL v2	No existe	No hay módulos de Cliente o Venta en el prototipo	Diseñado en artefactos externos al prototipo; no implementado ni probado en la API.

Fuentes: plan de pruebas, script de base de datos v2, implementación de cultivos y pruebas.

9. Nota de uso de IA

• Herramienta: GitHub Copilot en Visual Studio Code.

• Uso: revisar el repositorio frente a los requisitos del taller; redactar borradores de ADR, matriz de stack, plan de pruebas, registro de deuda técnica, matriz de riesgos, trazabilidad y esta nota; ampliar pruebas unitarias y proponer e implementar una consulta de cultivos por identificador.

• Contenido generado o modificado con asistencia de IA: documentación nueva en `docs/`, endpoint `GET /api/cultivos/{id}` y pruebas de validación/consulta individual.

• Decisión aportada por el estudiante: conservar las herramientas y el stack existentes en el proyecto.

• Revisión y cambios del equipo: a la fecha de esta nota, no se ha reportado una revisión o modificación adicional del equipo sobre los borradores. El equipo debe verificar los puntajes de familiaridad, estimaciones, IDs provisionales de historias, estados de riesgo y decisiones antes de presentar la entrega.

• Fuentes humanas utilizadas: repositorio AgroStock compartido por el equipo, modelos draw.io y `Requisitos entrega 2.pdf`.

La asistencia de IA no reemplaza la validación del equipo ni la correspondencia con el backlog y la matriz de riesgos originales del curso.

10. Documento consolidado y fuentes

Este PDF consolida los componentes disponibles de la entrega. Las cinco fuentes SVG están en `docs/modelos/`; falta incorporar la fuente visual del diagrama de clases. Las instrucciones del generador y sus dependencias están en `scripts/build_delivery_pdf.py` y `docs/requirements-pdf.txt`.

Para regenerarlo desde la raíz del repositorio:

```
py -m pip install -r docs/requirements-pdf.txt
py scripts/build_delivery_pdf.py
```

El prototipo es una API .NET autocontenida. El alcance y las diferencias respecto a los modelos están descritos en ADR-0001 y la matriz de riesgos; no se presenta como implementación completa de React/Laravel/MariaDB.